



Universidad de Concepción

Facultad de ingeniería

Sistemas Operativos

Tarea 1:

Implementación de una Shell simple

Docente: Cecilia Hernández

Integrantes:

- Josefa Arriagada [N°2025423812]
- Esteban Astete [N°2025446162]
- Vicente Carrasco [N°2025451514]
- Tomás Pizarro [N°2025435799]

Fecha de entrega: 28/09/2026

Introducción

El presente proyecto consiste en el diseño e implementación en C de miShell, un intérprete de comandos interactivo para sistemas operativos. El objetivo principal fue desarrollar una interfaz con capacidad para gestionar ciclo de vida de procesos en primer y segundo plano, redirigir descriptores de archivo, encadenar comandos mediante tuberías, manipular señales del sistema y consultar el estado del sistema a través de herramientas internas de monitoreo.

Arquitectura general

Main.c: Bucle principal, gestiona el prompt, la captura con readline, tokenización y enrutamiento hacia comandos internos o externos.

Shell.c: Lógica de tokenización, ejecución de comandos simples y manejo de segundo plano.

Pipes.c: Implementación de encadenamiento de N procesos y duplicación de descriptores.

Redirecciones.c: Implementación en la redirección de entrada y salida estándar mediante signos (>, >>, <).

Señales.c: Configuración de la tabla de señales, aislamiento de la Shell principal e integración de cosechador de zombies.

Job.h: Estructura de datos global para el registro y estado de los trabajos en segundo plano.

Pmon.h: Implementación de herramienta de monitoreo activo que permite observar los procesos en segundo plano.

Funcionalidades Principales Implementadas:

Tokenización y gestión de argumentos: La función tokenizar procesa la cadena cruda introducida por el usuario. A diferencia de un separador básico por espacios, utiliza un estado lógico interno para detectar comillas simples, permitiendo preservar espacios dentro de cadenas. Asimismo, detecta la presencia del carácter “&” al final del comando, activando la bandera correspondiente y removiendo dicho símbolo del arreglo final de argumentos para evitar fallos durante la invocación de `execvp`.

Ejecución de procesos y segundo plano: El primer plano (Foreground) funciona mediante la clonación del proceso con `fork()`; en el proceso padre se realiza un bloqueo mediante `waitpid()` hasta que el hijo finalice su ejecución. El segundo plano (Background) utiliza una bandera para verificar si el comando debe correr de forma asíncrona. En este caso, el proceso padre no invoca un `waitpid()` bloqueante, sino que registra la información del proceso (PID e ID) en el arreglo `lista_jobs` marcándolo como activo, imprime el identificador en un formato determinado y devuelve inmediatamente el control al prompt.

Tuberías encadenadas múltiples: El módulo `pipes.c` analiza el comando buscando delimitadores. Si existen N comandos, la función genera N-1 tuberías mediante `pipe()`. Para cada subcomando se realiza un `fork()`. En el proceso hijo, la entrada estándar se reasigna al extremo de lectura de la tubería anterior mediante `dup2()`, mientras que la salida estándar se reasigna al extremo de escritura de la tubería siguiente. Es fundamental cerrar todos los descriptores de pipe no utilizados en cada proceso. Si no se cierran, ocurre un escenario de bloqueo (deadlock): si un proceso hijo hereda un extremo de escritura abierto que nunca usa, el proceso lector del siguiente comando

nunca recibirá la señal de fin de archivo (EOF) y se quedará esperando datos indefinidamente.

Comandos internos: Los comandos internos se ejecutan en el contexto directo del proceso de la Shell sin separar nuevos procesos mediante `fork()` y `exec()`. Esto es estrictamente necesario porque atributos como el directorio de trabajo actual o la ejecución misma del programa pertenecen al entorno del proceso individual.

- **Cd:** Utiliza la llamada al sistema `chdir()`. Si se implementara mediante `fork()`, el proceso hijo cambiaría su propio directorio y moriría inmediatamente, dejando a la shell principal exactamente en el mismo directorio original. Además, se añadió un algoritmo de limpieza previa para remover comillas y soporta la navegación al directorio base del usuario cuando no cuenta con argumentos.
- **Jobs:** Recorre el arreglo `lista_jobs` filtrando y mostrando el listado ordenado de las tareas en segundo plano activas.
- **Exit:** Finaliza limpiamente el ciclo de ejecución de la Shell principal. Si operara sobre un proceso hijo, únicamente mataría a ese subprocesso y la shell continuaría ejecutándose con normalidad.
- **Pmon:** Invoca la rutina de monitoreo interactivo del sistema de manera directa.

Manejo de señales y gestión de procesos zombies: Un aspecto crítico fue el control del comportamiento de las señales para proteger la Shell sin afectar el funcionamiento de los subprogramas.

- **Protección de la Shell:** Mediante `sigaction`, la Shell configura las señales como `Ctrl+C` (`SIGINT`) para ser ignoradas (`SIG_IGN`). Esto evita que la Shell colapse accidentalmente si el usuario presiona combinaciones mientras interactúa con la línea de comandos.
- **Restauración de señales en procesos hijos:** Por defecto, los procesos hijos heredan los manejadores de señales de su padre. Si no se modificara la disposición heredada de la shell principal, comandos en primer plano como `sleep 30` heredarían el estado de ignorar `Ctrl+C`, volviéndose inmunes a las interrupciones del usuario. Por ello, dentro del proceso hijo creado por `fork()` y previo a la ejecución de `execvp()`, se restablece explícitamente la acción por defecto (`SIG_DFL`) para `SIGINT` en los comandos de primer plano.
- **Recolección asincrónica de procesos zombies:** Cuando un proceso en segundo plano termina, el sistema operativo envía una señal `SIGCHLD`. En Linux, las señales estándar no se encolan; si múltiples subprocessos mueren de forma casi simultánea, el kernel puede enviar una única señal. Para procesar esto correctamente, el manejador ejecuta `waitpid()` dentro de un ciclo `while` utilizando la bandera `WNOHANG`. Este ciclo asegura barrer y limpiar absolutamente todos los hijos terminados en una sola pasada de forma no bloqueante, evitando que

queden procesos zombies. Al recuperar el PID, la shell actualiza el estado en lista_jobs, reporta la terminación del comando y reimprime el prompt para no romper la presentación de la terminal.

Módulo de monitoreo: El comando interno pmon implementa una interfaz de inspección continua del sistema accediendo directamente al sistema de archivos virtual /proc.

Mecanismo de lectura e inspección: El módulo identifica carpetas cuyos nombres consisten en caracteres numéricos (PID) y lee sus archivos internos para extraer información técnica. Existe la condición de carrera donde un proceso monitoreado puede terminar justo entre dos lecturas de /proc, provocando que su carpeta desaparezca de forma repentina. La implementación maneja este caso validando de forma segura que el puntero de lectura no sea nulo (ej. if (archivo != NULL)). Si ocurre, se omite silenciosamente ese proceso en esa iteración, previniendo fallos de segmento (Segmentation Fault) o la caída total de la shell.

- Muestreo configurable: La función recibe opcionalmente la tasa de refresco en segundos. Renderiza la información en una tabla calculando el uso global de CPU y memoria RAM, borrando la pantalla mediante secuencias de escape y pausando el hilo de ejecución hasta que el usuario decida salir.

Bonus e innovaciones: Se cambió el mecanismo fgets por la librería readline, permitiendo:

- Navegación e historial: Almacena dinámicamente cada comando ingresado, habilitando la navegación interactiva mediante las flechas direccionales (arriba y abajo).
- Corrección de ancho de cursor: Dado que el prompt utiliza colores para mejorar la interfaz, se envolvieron los códigos de color entre las secuencias de delimitador \001 y \002. Esto evita que la librería calcule erróneamente la longitud en caracteres del prompt y desalinee la posición del cursor al editar texto largo.
- Preservación del nombre completo: Al detectar que guardar un pipeline asíncrono truncaba cosméticamente los argumentos en pantalla, se reconstruye la cadena completa ingresada, garantizando que la salida del comando jobs muestre la línea de comandos íntegra.

Instrucción de compilación: Compilar la Shell: make | Ejecutar la Shell: ./mishell | Limpiar ejecutables y otros: make clean. Antes de iniciar se debe contar con las librerías correspondientes.

Test: las pruebas se realizaron con casos borde, para verificar la funcionalidad de cada requerimiento pedido.

Enlace del repositorio: https://github.com/EstebanAsth/TareaSO_1